

A Deep Dive into the Hyperledger Sawtooth Architecture

This article gives the reader an insight into the Hyperledger Sawtooth architecture. It is targeted at open source technology enthusiasts who have an interest in blockchain technologies and a working knowledge of the Hyperledger Sawtooth project.



One goal of a distributed ledger like Sawtooth — indeed, its defining goal—is to distribute a ledger among participating nodes. The ability to ensure a consistent copy of data amongst nodes in the Byzantine consensus is one of the core strengths of blockchain technology. Sawtooth represents the state for all transaction families in a single instance of a Merkle-Radix tree on each validator. The process of block validation on each validator ensures that the same transactions result in the same state transitions, and that the resulting data is the same for all participants in the network.

Global state

The state is split into name spaces, which offer the flexibility to transaction family authors to define, share and reuse global state data between transaction processors. Sawtooth uses an addressable Merkle-Radix tree to store data for transaction families. Let's break that down — the tree is a Merkle tree because it is a copy-on-write data structure, which stores successive node hashes from leaf-to-root upon any changes to the tree.

The tree is an addressable Radix tree, because addresses uniquely identify the paths to leaf nodes in the tree where information is stored. An address is a hex-encoded 70-character string representing 35 bytes. In the tree implementation, each byte is a Radix path segment, which identifies the next node in the path to the leaf containing the data associated with the address. The address format contains a 3-byte (6-hex character) name space prefix, which provides 224 (16,777,216) possible name spaces in a given instance of Sawtooth. The remaining 32 bytes (64 hex characters) are encoded based on the specifications of the designer of the name space, and may include schemes for sub-dividing further, distinguishing object types, and mapping domain-specific unique identifiers into portions of the address.

The journal

The journal is the group of validator components that work together to handle batches and proposed blocks. These components are responsible for completing published blocks, publishing batches into blocks to extend the chain, and validating proposed blocks to determine if they should be considered for the new chain head.